CSE 151B/251B Final Report

Vy Dang
Department of Computer Science & Engineering
University of California, San Diego
kid002@ucsd.edu

Eric Wang
Department of Computer Science & Engineering
University of California, San Diego
ejw001@ucsd.edu

Kilhoon Kim
Department of Computer Science & Engineering
University of California, San Diego
kik012@ucsd.edu

Abstract

With the rise of the Language Model and its popularity, many large-scale models have been developed in order to boost its learning and reasoning capabilities. However, it comes with an abundance of size that strangles lots of resources on the server's hardware and users who want to run local LLMs on their devices. This paper will focus on a smaller-sized model, Alibaba's Qwen3 with 4-billion learnable parameters, and aim to maximize the boundary of LLM Mathematical Reasoning, prioritizing "Quality over Quantity". Our final model ended up scoring 21st in the Kaggle competition. Link to our GitHub.

1 Introduction[2 pt]

Problem Definition.[0.25 pt] The task is to fine-tune a small decoder language model Alibaba Qwen3, with only 4 billion learnable parameters, to maximize its mathematical reasoning success rate using a dataset more than 1000 questions mathematical questions and answers labelled.

Problem Significance.[0.25 pt] If we can maximize the success rate with a smaller and lightweight decoder model, it will allow others with constrained hardware and financial resources to run this model locally and still achieve great performance and results. Better yet, even deploying this light-weight model consumes less power on GPUs' resources, burns less water for cooling, and improves data center efficiency.

Technical Challenge.[0.5 pt] Since we have only around 1000 data points, k folds may have unequal difficulty distributions, as the questions can vary from high school to college level math. The biggest challenges we are facing are about the training data we have of roughly 1000, the model's design of only 4-billion learnable parameter (comparing to SOTA 70B) that might limits it's knowledge, application of LoRA on Qwen3-4B, or even Full Parameter Fine-Tuning and RAG, and finding a baseline and benchmarking methods to succeed.

Contributions. [1 pt] Infrastructure: Uses uv for fast dependency management and vLLM or Transformers for model hosting.

Preprint. Under review.

Quantization: Employs INT4/INT8 quantization via bitsandbytes to fit the 4B parameter model into GPU environments

Prompting: Utilizes a Zero-Shot approach with specific system prompts that instruct the model to wrap final answers in $\{\}$ tags.

Evaluation: Employs regex for MCQ extraction and a symbolic Judger for free-form questions to check for mathematical equivalence beyond literal string matching.

Weakness of starter code:

- Zero-Shot limitations:
  The model is asked to solve complex problems immediately without seeing any examples of the expected reasoning steps or formatting.
- Quantization Loss:
  using 4-bit or 8-bit quantization helps with memory but can reduce the model's accuracy and logical consistency in multi-step math problems.
- Rigid Extractions:
  The scoring logic relies heavily on the model correctly using the $\backslash$boxed$\{\}$ command; if the model deviates from this specific format, the answer is often marked incorrect even if the logic is right.
- Basic Prompting:
  The build_prompt function only combines a generic system message with the raw question, missing out on more advanced techniques that guide the model's thinking process.

Our main contributions are:

- Contribution 1: Few-Shot Example Injection
  We modified the build_prompt function to inject a new EXAMPLES block before the actual question. By showing the model a completed MCQ and a Free-form example, the model now has a "template" to follow. This reduces the "Learning from Scratch" problem identified in the starter code.
- Contribution 2: Strict Output Control
  We updated the SYSTEM_PROMPT_MATH and SYSTEM_PROMPT_MCQ to be significantly more demanding. The new prompts explicitly tell the model to output ONLY the final answer in the box and forbid "extra commentary." This directly fixes the "Extra Talking" drawback and makes the scoring much more reliable.

2 Related Work[1 pt]

Related works topic 1. Wang et al. [1] introduced a family of tiny reasoning models, Tina, which achieve high cost-efficiency and strong reasoning abilities through the use of parameter-efficient reinforcement learning via LoRA (Wang et al., 2025).

3 Methods[4 pt]

Problem Setting.[1 pt] We define the task of mathematical reasoning as a sequence generation problem. Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ where $N \approx 1000$, each input x_i represents a mathematical query in natural language, and y_i represents the corresponding solution. Our goal is to optimize the parameters of the Alibaba Qwen3 decoder model (4B parameters) to maximize the probability of generating the correct reasoning path. To measure correctness, we will inject the prompt for the model and tell the model to boxed the final answer, wait for it to generate, before checking for the SINGULAR boxed final answer in the dataset.

Idea Summary.[1 pt] We decided to apply Prompt Engineering as that is the cheapest method to get at least 5-10% bump in accuracy, such as telling the model to stay consistent with rounding up to 3 digits, and box the final answer. Follow up with that is few-shot prompting by injecting the

prompts with examples of the answering format, to reduce false negative errors. With that done, we always invoke the model into "thinking" mode and to says out what is it thinking and doing (chain-of-thoughts). Lastly, we implemented majority voting, which is a method where for the same problem, do the same problem $n - times$, and saved all of those n number of answers, and pick out the most common one as the final answers for consistency.

Considerations. The model we are working with only has 4-billion parameters, so our biggest consideration is the lack of "world knowledge" during the pre-training period; comparing to other SOTA where they have 70B+, our model has to compress trillions of training tokens into 4-billion will ended up with a weaker reasoning in niche area. Therefore we also considering implementing Full Parameter Fine-Tuning to shift Qwen3-4B to exceptional math reasoning based on our training set, and combine with retrieval-augmented generation (RAG) to allow the model look up on the internet during inference time to solve the "memory" problem on a 4B parameter, allowing it to access external information.

Method Description.[2 pt] We did not dive into training and validation data due to a reason of lack of computational power to implement k-fold cross validation and it's not appropriate for mathematical reasoning. At first, we attempted 8-folds: 6 training, 1 validation, 1 testing. But it is very unfeasible to perform cross validation 8 times, while the training progress took up all of our GPU resources and RAM and crashed a multiple times back in Week 6 without giving any positive results. Therefore, we decided that we will not implement cross validation or any training data. Besides, Qwen3-4B is already a solid reasoning model for mathematics, and mathematics are all logical sense, so by training it on the training dataset, will most likely overfitting the model, to our conclusion.

4 Experiments[6 pt]

4.1 Baselines[0.5 pt]

We compare with the following baselines:

- **Baseline A**: Random output: randomly selects an answer for MCQs and gives a placeholder for FRQs.
- **Baseline B**: Qwen3-4B-Thinking-2507: Dense transformer with 36 layers, grouped query attention, 4B parameters.

4.2 Evaluation[0.5 pt]

We evaluate our model through timing and accuracy. We check total accuracy and accuracy for FRQs and MCQs.

4.3 Implementation details[2 pt]

The goal of this section (together with other relevant parts of your paper) is to ensure full reproducibility without having to check your code. Include figures and data to support your explanation. Some relevant points to include:

- Our inference pipeline was executed on a single GPU. The specific model used was Nvidia L40S. Since we used a pre-trained model. Training time per epoch is not applicable. The inference process is optimized with vllm library, which is designed for high throughput LLM serving.
- Our implementation is centered around the Qwen3-4B-Thinking-2507 model, a 4-bilion parameter model optimized for chain of thought reasoning. Key hyperparameters and design choices for our strategy are described below:

  A. Inference Engine: we used vlm for LLM inference, enabling us to process the dataset efficiently

  B. Self-Consistency: To improve the reliability of our answers, we employ self-consistency. For each question, we generate n=5 independent responses. The final answer is determined by a majority vote on the content extracted from the `\boxed{}` block in each generation.

This mitigates the risk of occasional reasoning errors or hallucinations. We found the best parameters for Self-Consistency was n=5. However, after the leaderboard for the private set test was published, n=4 model got better accuracy. We also tested with n=3 but we found that n=3 was not enough to gain an accuracy for the public test set.

C. Prompt Engineering: We developed distinct system prompts for multiple choice questions and free from questions. The prompts instruct the model on the required step-by-step reasoning process and the matching with the final answer format. We also provided two few-shot examples within the prompt to guide the model's behavior.

The following parameters were used for generating responses with vllm:

Parameter	**Value**
temperature	0.6
top_p	0.95
top_k	20
repetition_penalty	1.0 (No penalty)
max_new_tokens	38912
max_model_len	16384

Table 1: Hyperparameter Configuration

Choice of Hyper-parameters: To balance correctness and diversity, we closely followed the hyperparameter configuration recommended for the Qwen model on its official repository (https://huggingface.co/Qwen/Qwen3-4B).

A temperature of 0.6 was chosen to make the model's output more deterministic and focused, which is crucial for mathematical accuracy, while still allowing for slight variations in the reasoning path across the 5 generated samples.

The `top_p` and `top_k` values further constrain the sampling to likely tokens, reinforcing factual correctness.

The choice of $n = 5$ for self-consistency was a pragmatic trade-off between the significant accuracy gains it provides and the increased computational cost of generating multiple responses per question. These final values were selected based on general best practices for reasoning tasks and were confirmed to perform well on a small, internal validation set. The primary motivation was to maximize the correctness and robustness of the final answer.

4.4 Results[2 pt]

Our results are testing our model based on public set, in the Test column, the number is how many questions was running on specific model.

Table 2: Model Implementations and Performance Comparison

Implementations	Test	MCQ	Free-form	Overall	Time taken
Baseline + Few shots + chain-of-thoughts	10	66.67%	57.14%	60.00%	06:20% @ RTX A30
Baseline + Few shots + chain-of-thoughts	100	71.05%	53.23%	60.00%	1:36:33 @ RTX A30
Baseline + SFT + LORA	100	34.21%	9.68%	19.00%	H100 20GB
Baseline + Prompt Engineering + thinking, slight hyperparameter tweak	200	57.35%	62.12%	60.50%	$\sim$17m @ L40S
Baseline + Prompt Engineering + thinking, slight hyperparameter tweak, majority voting (n = 4)	200	70.59%	64.39%	66.50%	1.5 hr @ L40S
Baseline + QLoRA + SFT	100	–	–	18.00%	20min @ L40s
Baseline + QLoRA + SFT + Prompt Injection + FewShot	100	–	–	30.00%	15min @ L40s
Baseline + Prompt Engineering + FewShot + Majority voting (n=5)	963 (private)	–	–	69.90%	12hours @ L40s
Baseline + Prompt Engineering + FewShot + Majority voting (n=4)	963 (private)	–	–	69.20%	7hours @ L40s

4.5 Ablations[1 pt]

The results from Table 1 allow for several direct comparisons to isolate the contributions of specific design choices.

1. Impact of Majority Voting: Isolating the contribution of majority voting yields the most consistent performance gains.
   A. Comparing the Baseline + Prompt Engineering + thinking, slight hyperparameter tweak implementation with and without majority voting (n=4) on the 200-sample test set shows an overall performance increase from 60.50 to 66.50.
   B. Further ablating the ensemble size (n=4 vs. n=5) on the larger 963-sample private test set shows a marginal improvement from 69.20 to 69.90. This indicates that while majority voting is highly effective, increasing the voting pool beyond 4 yields diminishing returns.
2. In-Context Learning vs. Parameter-Efficient Fine-Tuning (PEFT): Comparing training-free methods against fine-tuning reveals a stark contrast in effectiveness for this specific task.
   A. The Baseline + Few shots + chain-of-thoughts approach achieved an overall score of 60.00.
   B. Replacing the prompting strategy with fine-tuning (Baseline + SFT + LORA and Baseline + QLoRA + SFT) drastically reduced performance to 19.00 and 18.00, respectively. This suggests the fine-tuning implementations likely suffered from suboptimal hyperparameters, inadequate training data volume, or catastrophic forgetting, making the base model's inherent reasoning capabilities (unlocked via prompting) the superior choice.
3. Impact of Prompt Enhancements on Weakened Models: While the QLoRA/SFT models performed poorly, isolating the addition of prompting techniques demonstrates their restorative value.

A. Comparing Baseline + QLoRA + SFT (Overall: 18.00) to Baseline + QLoRA + SFT + Prompt Injection + FewShot (Overall: 30.00) shows a +12.00 point gain. Even when the underlying model weights are detrimentally altered, structured context (FewShot) significantly recovers performance.

5 Discussion[2 pt]

Achievements. [0.5 pt] We achieved a 0.633 accuracy on the private test data for the Kaggle competition. This was quite notable, since the highest score achieved was 0.774, and we landed in 21 out of 105 entries. We mainly contributed to model development.

Bottlenecks and Mitigation. [0.5 pt] Our biggest bottleneck is the 4B parameter restriction of the model. We used few-shot injection and increased the temperature.

At one point, when we were attempting to implement Supervised Fine-tuning and (Quantized) Low Rank Adaptation on top of Systems prompting, the model's performance diminished from average 60% of baseline to 19%. That did add a couple days of time, but at least we figured out it could not work.

For the biggest bottleneck of this project was working environment dependencies and lack of GPU resources for training purposes. We constantly get into troubles of having unreliable GPU resources from the UCSD's GPU clusters. Most overhead time goes into troubleshooting working environment compatibilities, disconnect mid-training after 2 hours in, GPU cluster session stopped and killed our process, etc. We were also using external GPU providers, however, it does come with a pricey costs. So we figured out that by training at midnight is when we can allocate enough GPU cluster resources, and begin our experiment on a small data subset (100, 200 out of 1000 questions), keep track of our implementations and hyper-parameters, and find the best performance model, before scaling it up for a full pass.

Lessons Learned. [0.5 pt]

- Lessons Learned:
  A. Selecting Problems and Literature Survey: Working with Alibaba's Qwen3-4B model highlighted the challenges of applying a small, 4-billion parameter model to complex mathematical reasoning tasks. Because the model naturally lacked the "world knowledge" of 70B+ state-of-the-art models, our focus had to shift toward maximizing its inherent logical capabilities rather than expanding its raw knowledge base.
  B. Technical Implementation: A major takeaway was that advanced fine-tuning techniques do not automatically yield better results. Implementing Supervised Fine-Tuning (SFT) and Quantized Low-Rank Adaptation (QLoRA) actually diminished our model's performance from an average 60% baseline down to roughly 19%. Conversely, we found that focusing heavily on Prompt Engineering—specifically through Few-Shot Example Injection and Strict Output Control—provided a much cheaper and highly reliable accuracy bump. Coupling these prompts with a self-consistency strategy (majority voting with n=4 or n=5) proved to be the most robust method for mitigating reasoning errors.

- Future Improvements: If given more time, we would focus on the following aspects of project development:
  A. Implement Retrieval-Augmented Generation (RAG): As noted in our initial considerations, the 4B parameter model suffers from a "memory" problem due to its inability to compress trillions of training tokens. We would dedicate time to building a RAG pipeline, allowing the model to access external information during inference to compensate for its limited pre-training knowledge.
  B. Stabilize Infrastructure and Dependencies: A massive amount of overhead was lost to troubleshooting working environment dependencies and recovering from external GPU disconnects. We would prioritize setting up more reliable, isolated environments (perhaps utilizing more stable, albeit pricey, external GPU providers earlier on) to ensure uninterrupted training and validation.
  C. Investigate Fine-Tuning Failures and Cross-Validation: Due to GPU crashes and RAM

limits in Week 6, we had to abandon 8-fold cross-validation and deep training on the dataset. With more stable compute, we would rigorously debug our SFT and QLoRA pipelines to understand exactly why they caused performance degradation, and we would run proper k-fold cross-validation to better map the unequal difficulty distributions of the dataset.

Team Responsibilities. [0.5 pt]

- Eric: Taken CSE151A and CSE160, have surface-level knowledge of all the methods we need to implement our final model. Helped get starter code working on Colab, and fine-tuned hyperparameters. Filled out most of the docs.
- Kilhoon: Taken CSE151A and CSE160, have basic knowledge of machine learning, implemented few shot injection and modifying prompt to strict output control. Implemented LoRA + SFT. Help get the starting code working, set up the development environment, run the experiments, and record results.
- Vy: Took CSE152A: Intro to CV and CSE156: NLP, have a high-level and low-level implementation understanding of Transformers architectures and decoder models. Have some experience in LLM's security and attempts to jailbreak them (locally). Found Wang's scholarly paper and, by combining it with CSE156's knowledge, came up with architectural ideas, which set the foundation for what to do next. Filled out most of the docs.

LLM Usage

To help set up the developing environment for the starter code, LLM was used in order to debug and install dependencies. LLM was also used to help with formalizing and writing the docs in LaTeX.

References

[1] Shangshang Wang, Julian Asilis, Ömer Faruk Akgül, Enes Burak Bilgin, Ollie Liu, and Willie Neiswanger. Tina: Tiny reasoning models via lora. *arXiv preprint arXiv:2504.15777*, 2025. doi:10.48550/arxiv.2504.15777.